

Day 7: Bash Scripting and IP Sweep

Today, I started learning the basics of bash scripting and how it can be used to automate simple tasks in Linux. My focus was on understanding how commands I usually type in the terminal can be placed inside a script and reused, instead of typing them manually every time. This helped me understand how automation works at a beginner level.

Understanding Bash Scripting

Bash scripting, the way I am learning it, is about writing a sequence of terminal commands inside a text file. These commands can then be executed together as a script, which makes repetitive tasks easier and faster to perform.

Saving a Ping Result into a File

One of the first commands I practiced with was ping. If I want to send only one ICMP packet, I use the `-c 1` option. The `-c` option represents the count, which means how many ping requests I want to send. Using output redirection, I can save the result into a text file instead of printing it on the screen.

```
ping 1.1.1.1 -c 1 > IP.txt
```

```
cat IP.txt
```

Filtering Output Using Pipes

To work with the output inside the file, I learned how to use pipes. The pipe symbol takes the output of one command and sends it as input to another command. I used `grep` to search for specific output, such as '64 bytes', which indicates that a host is reachable and responding to the ping request.

```
cat IP.txt | grep "64 bytes"
```

Extracting and Cleaning Data

After filtering the output, I used the `cut` command to extract a specific field from the result. The delimiter option allows me to define how the text is split, and the field option lets me choose which part I want. I then used the `tr` command to remove unwanted characters from the output, such as colons.

```
cat IP.txt | grep "64 bytes" | cut -d ' ' -f 4 | tr -d ":"
```

Creating the Bash Script

To put everything together, I created a bash script file using Mousepad. At the top of the file, I added the shebang line to tell Linux that the script should be executed using bash. I also learned how to pass an IP address as an argument and check if the user forgot to provide it using an if statement.

```
mousepad IPSweep.sh
```

```
#!/bin/bash
```

```
./IPSweep.sh <IP address>
```

```
if [ "$1" == "" ]  
then  
echo "You forgot an IP address"  
echo "Syntax: ./IPSweep.sh <IP address>"
```

Using a Loop for IP Sweep

Finally, I used a loop to scan a range of IP addresses from 1 to 254. This allowed the script to ping multiple IPs efficiently. Running the ping commands in the background made the scan faster and more effective.

```
else  
for IP in $(seq 1 254);  
do  
ping -c 1 $1.$IP | grep "64 bytes" | cut -d " " -f 4 | tr -d ":" &  
done  
fi
```

This day helped me understand how bash scripting can be used to automate tasks and perform simple network scanning. It gave me more confidence working with scripts and reinforced the importance of learning Linux fundamentals as part of my cybersecurity journey.